



UNIVERSITE DE FIANARANTSOA
ÉCOLE NATIONALE D'INFORMATIQUE

RAPPORT DE PROJET EN TECHNOLOGIE DE PYTHON LE EN TROISIEME
ANNEE DE LICENCE PROFESSIONNELLE

Mention : Informatique

Parcours : Génie Logiciel et Bases de données

intitulé :

ELABORATION D'UNE PLATEFORME DE RESERVATION INTELLIGENTE

Présenté par :

- Monsieur RAZAFINIASY Handi Miguel 3339
- Monsieur FANILONTSOA Vetsotiana Sarobidy 3181
- Monsieur ANDRIAMIHAJA Mahafaly Odilon 3349

Rapporteurs :

- Docteur RALAIVAO Jean Christian , Maitre de conference

Année Universitaire : 2025-2026

SOMMAIRE

SOMMAIRE.....	I
REMERCIEMENTS.....	III
LISTE DES FIGURES.....	IV
LISTE DES TABLEAUX.....	V
LISTE DES ABREVIATIONS.....	VI
INTRODUCTION.....	1
PARTIE I. DESCRIPTION DU PROJET	3
1.1 Formulation.....	4
1.2 Objectifs et besoins des utilisateurs.....	4
1.2.1 Objectifs généraux.....	4
1.2.2 Besoins spécifiques des utilisateurs.....	5
1.3 Résultats attendus.....	5
2.1 Technologies utilisées	8
2.2 Installation et configuration des outils	8
2.2.1 Installation de Python.....	9
2.2.2 Installation des bibliothèques Python	9
2.2.3 Installation et configuration de PostgreSQL	9
2.3 Structure du projet.....	10
2.4 Présentation du code essentiel.....	11
2.4.1 Connexion à la base de données	11
2.4.2 Configuration	12
2.4.3 Sécurité	13
2.4.4 Application principale	14
2.4.5 Modèle utilisateur (app/models/user.py)	15
3.1 Page d'accueil	16
3.2 Page de connexion / inscription.....	17
3.3 Espace professionnel (hôte).....	18

3.3.1 Tableau de bord.....	18
3.3.2 Gestion des réservations	19
3.3.3 Gestion des disponibilités	20
3.3.4 Notifications	21
3.3.5 Profil public.....	22
3.4 Espace client.....	23
3.4.1 Tableau de bord.....	23
3.4.2 Mes réservations	24
3.4.3 Recherche de professionnels	25
3.4.4 Page de réservation.....	26
3.4.5 Assistant IA	27
3.4.6 Notifications	28
3.5 Espace administrateur.....	29
3.5.1 Tableau de bord.....	29
3.5.2 Gestion des utilisateurs.....	30
CONCLUSION	31
GLOSSAIRE.....	VII
ANNEXES	VIII

REMERCIEMENTS

En premier lieu, je tiens à remercier Dieu tout puissant de m'avoir donné la force, la santé et l'enthousiasme nécessaire pour la réalisation de ce stage.

J'exprime aussi mes profonds remerciements à :

- Monsieur HAJALALAINA Aimé Richard, Docteur HDR, Président de l'Université de Fianarantsoa, qui a bien organisé l'année universitaire.
- Monsieur MAHATODY Thomas, Professeur à l'Université de Fianarantsoa et Directeur de l'École Nationale d'Informatique qui nous a donné l'opportunité de partir en stage afin de consolider mes connaissances et d'en développer de nouvelles.
- Madame RAKOTOMALALA Adriana Veronirina Patricia , Directeur Général de l'ANALOGH , pour nous avoir accueilli au sein de l'organisme et nous avoir permis d'y effectuer notre stage.
- Monsieur RALAIVAO Jean Christian, Maître de conférences, responsable de mention informatique, de votre engagement à nous fournir une formation de qualité qui nous préparé à notre futur carrier.
- Madame RATIANANTITRA Volatiana Marielle, Maître de conférences, responsable de parcours Génie Logiciel et Base de Données , de votre engagement à nous fournir une formation de qualité qui nous préparé à notre future carrière.
- Tous les enseignants de l'Ecole Nationale d'Informatique pour les connaissances qu'ils nous ont transmis durant notre formation théorique.
- Finalement, je tiens à remercier les membres de ma famille de m'avoir soutenu que ce soit moralement, matériellement et financièrement tout au long de ce stage.

LISTE DES FIGURES

Figure 1 : Structure du projet.....	10
Figure 2 : Connexion à la base de données.....	11
Figure 3 : Configuration de l'application	12
Figure 4 : Sécurité et authentification.....	13
Figure 5 : Application principale.....	14
Figure 6 : Modèle utilisateur.....	15
Figure 7 : Page d'accueil.....	16
Figure 8 : Page de connexion	17
Figure 9 : page d inscription	18
Figure 10 : Tableau de bord professionnel	18
Figure 11 : Gestion des réservations	19
Figure 12 : Gestion des disponibilités.....	20
Figure 13 : Notifications du professionnel	21
Figure 14 : Profil public du professionnel	22
Figure 15 : Tableau de bord client.....	23
Figure 16 : Mes réservations	24
Figure 17 : Recherche de professionnels.....	25
Figure 18 : Page de réservation	26
Figure 19 : Assistant IA	27
Figure 20 : Notifications du client.....	28
Figure 21 : Tableau de bord administrateur	29
Figure 22 : Gestion des utilisateurs	30

LISTE DES TABLEAUX

Tableau 1 : Technologies utilisées.....	8
---	---

LISTE DES ABREVIATIONS

API : Application Programming Interface
ASGI : Asynchronous Server Gateway Interface
CORS : Cross-Origin Resource Sharing
CSS : Cascading Style Sheets
CSV : Comma-Separated Values
ENI : École Nationale d'Informatique
HDR : Habilitation à Diriger des Recherches
IA : Intelligence Artificielle
JSON : JavaScript Object Notation
JWT : JSON Web Token
ORM : Object-Relational Mapping
RAG : Retrieval-Augmented Generation
REST : Representational State Transfer
SDK : Software Development Kit
SQL : Structured Query Language
UX/UI : User Experience / User Interface

INTRODUCTION

À l'ère du numérique, l'informatique occupe une place essentielle dans tous les secteurs d'activité, en améliorant l'efficacité, la rapidité et la fiabilité des traitements de l'information. La gestion des rendez-vous et des disponibilités des professionnels demeure pourtant souvent manuelle ou peu structurée, entraînant des erreurs de planification, une perte de temps et une organisation inefficace. Il devient donc nécessaire de disposer d'un système informatisé pour faciliter la mise en relation entre professionnels et clients et optimiser les réservations en ligne.

La problématique centrale de ce projet est la suivante : **comment concevoir une application web permettant d'automatiser la gestion des disponibilités et des réservations, tout en répondant aux besoins des professionnels et des clients, et en optimisant les processus de prise de rendez-vous ?**

Pour y répondre, nous avons développé une plateforme de réservation intelligente offrant trois espaces distincts : un espace professionnel pour gérer disponibilités, tarifs et exceptions ; un espace client pour rechercher et réserver des créneaux ; et un espace administrateur pour superviser les utilisateurs et les statistiques. L'application intègre également un assistant virtuel basé sur l'intelligence artificielle (Google Gemini) qui guide les clients dans leur recherche en langage naturel.

La solution repose sur une architecture moderne : un backend Python avec FastAPI, une base de données PostgreSQL, et un frontend React avec Tailwind CSS. La sécurité est assurée par JWT et le hachage des mots de passe. Le développement a suivi une démarche structurée en trois phases : analyse des besoins, conception, puis réalisation.

Le présent rapport est organisé en trois parties. La première présente l'environnement technique du projet, les outils utilisés et la structure de l'application. La deuxième traite de

l'analyse des besoins et de la conception du système. La troisième expose la réalisation concrète de la plateforme, suivie d'une conclusion générale et des perspectives d'évolution.

PARTIE I. DESCRIPTION DU PROJET

1.1 Formulation

La plateforme de réservation intelligente est une application web complète et moderne qui a été conçue dans le but de faciliter la mise en relation entre des professionnels de divers secteurs d'activité (tels que des coachs sportifs, des consultants, des coiffeurs, des thérapeutes, etc.) et des clients particuliers ou professionnels qui souhaitent prendre rendez-vous en ligne de manière simple, rapide et efficace.

Le projet repose sur une architecture robuste combinant un backend API développé avec FastAPI et une interface utilisateur dynamique construite avec React. L'ensemble est soutenu par une base de données PostgreSQL pour assurer la persistance des données. La plateforme intègre également des services externes tels que Google Gemini pour l'assistant intelligent et un système d'envoi d'e-mails pour les notifications et confirmations.

L'objectif principal de cette plateforme est de digitaliser entièrement le processus de prise de rendez-vous, en éliminant les échanges fastidieux par téléphone ou par e-mail, et en offrant aux professionnels ainsi qu'aux clients un espace centralisé où toutes les informations relatives aux disponibilités et aux réservations sont accessibles en temps réel. Grâce à une interface épurée et intuitive, chaque utilisateur peut naviguer facilement dans son espace personnel et accomplir ses tâches sans difficulté.

En plus de la gestion classique des rendez-vous, le projet se distingue par l'intégration d'un assistant virtuel propulsé par l'intelligence artificielle de Google Gemini. Cet assistant permet aux clients de formuler des demandes en langage naturel, comme par exemple « Je recherche un coach sportif à Lyon », et de recevoir une réponse pertinente basée sur les professionnels inscrits sur la plateforme. Cette fonctionnalité améliore considérablement l'expérience utilisateur en rendant la recherche d'un professionnel plus naturelle et plus accessible.

Le développement de cette plateforme a nécessité une réflexion approfondie sur les besoins des utilisateurs finaux, la conception d'une architecture évolutive, la mise en place de mécanismes de sécurité fiables, et l'optimisation de l'interface pour offrir une expérience cohérente sur tous les appareils, qu'il s'agisse d'un ordinateur de bureau, d'une tablette ou d'un smartphone.

1.2 Objectifs et besoins des utilisateurs

1.2.1 Objectifs généraux

- **Autonomie des professionnels** : Permettre aux professionnels de gérer leurs disponibilités de manière autonome et flexible. Chaque professionnel peut définir ses horaires de travail pour chaque jour de la semaine, choisir la durée des créneaux qu'il souhaite proposer, et même fixer un prix pour ses services. Il peut également créer des exceptions pour bloquer certaines dates, par exemple pendant les vacances ou les jours fériés, afin de ne pas recevoir de réservations pendant ces périodes.
- **Simplification du parcours client** : Simplifier le parcours du client. Lorsqu'un client souhaite prendre rendez-vous, il peut rechercher un professionnel en utilisant des filtres avancés comme le type de service ou la localisation. Une fois qu'il a trouvé un professionnel, il peut consulter son profil public, voir ses créneaux disponibles sur plusieurs jours, et choisir le moment qui lui convient le mieux. Si le client est connecté à son compte, ses informations personnelles sont automatiquement pré-remplies dans le formulaire de réservation, ce qui rend le processus encore plus rapide.
- **Outil de supervision administrative** : Fournir un outil de supervision aux administrateurs. L'administrateur de la plateforme peut consulter une vue d'ensemble de tous les utilisateurs

inscrits, voir leur rôle (administrateur, professionnel, client), modifier ces rôles si nécessaire, et supprimer les comptes qui ne sont plus actifs. Il dispose également d'un tableau de bord global avec des statistiques sur le nombre d'utilisateurs, le nombre de réservations, et les revenus générés par la plateforme.

1.2.2 Besoins spécifiques des utilisateurs

Pour les professionnels (hôtes) :

- Disposer d'un espace personnel sécurisé pour gérer leurs informations et leurs disponibilités.
- Pouvoir créer plusieurs règles de disponibilité récurrentes.
- Définir un prix et un libellé pour chaque type de prestation.
- Bloquer des journées entières ou des plages horaires particulières.
- Recevoir des notifications immédiates lorsqu'une réservation est effectuée ou annulée.
- Consulter des statistiques détaillées sur leur activité : nombre de réservations, taux d'occupation, revenus.
- Exporter la liste de leurs réservations au format CSV pour une utilisation externe.

Pour les clients :

- Créer un compte en choisissant leur type de profil (client ou professionnel).
- Rechercher un professionnel selon des critères précis (service, localisation).
- Voir les disponibilités en temps réel et réserver un créneau en quelques clics.
- Ne pas avoir à saisir leurs coordonnées à chaque réservation s'ils sont connectés.
- Annuler une réservation s'ils ne peuvent pas honorer le rendez-vous.
- Utiliser un assistant virtuel pour trouver rapidement un professionnel adapté à leur besoin.

Pour l'administrateur :

- Gérer les comptes utilisateurs (changement de rôle, suppression).
- Visualiser des statistiques globales pour suivre l'évolution de la plateforme.
- S'assurer que les données sont cohérentes et que les utilisateurs respectent les règles.

1.3 Résultats attendus

À l'issue du développement de cette plateforme, plusieurs résultats concrets doivent être observés :

- **Système entièrement fonctionnel** : Le système doit être entièrement fonctionnel et permettre à un utilisateur de s'inscrire, de se connecter, de créer des disponibilités, de réserver un créneau, et de recevoir une confirmation par e-mail sans intervention manuelle. La chaîne complète de réservation doit être automatisée et fiable.
- **Expérience utilisateur (UX/UI) fluide** : L'expérience utilisateur doit être fluide et agréable. Grâce à l'utilisation de Tailwind CSS, les pages sont conçues selon un design moderne et épuré, avec une palette de couleurs cohérente et une typographie lisible. Les boutons, formulaires et cartes sont uniformes, ce qui renforce la crédibilité et la facilité d'utilisation.
- **Gestion dynamique des disponibilités** : La gestion des disponibilités doit être précise et tenir compte des fuseaux horaires. Les créneaux proposés aux clients sont calculés dynamiquement en fonction des règles définies par le professionnel, des exceptions éventuelles, et des réservations déjà effectuées. Cela garantit qu'un même créneau ne peut pas être réservé deux fois.

- **Automatisations des notifications et e-mails** : Les notifications et e-mails doivent être envoyés automatiquement pour informer les parties concernées. Par exemple, lorsqu'un client réserve un créneau, le professionnel reçoit immédiatement une notification dans son espace personnel et un e-mail de confirmation est envoyé au client. Si une réservation est annulée, les deux parties sont prévenues.
- **Assistant IA performant** : L'assistant IA doit être capable de répondre à des questions simples et complexes en se basant sur les données réelles des professionnels. Par exemple, si un client demande « Quels sont les coachs disponibles à Paris ? », l'assistant analyse les profils et retourne une réponse pertinente avec les noms, les services et les localisations correspondants.
- **Sécurité et contrôle d'accès** : La plateforme doit être sécurisée. Les mots de passe sont hachés avec bcrypt, l'authentification repose sur des jetons JWT, et les accès aux différentes pages sont contrôlés selon le rôle de l'utilisateur. Un client ne peut pas accéder à l'espace administrateur, et un professionnel ne peut pas modifier les comptes des autres utilisateurs.

En résumé, les résultats attendus sont : une application de réservation complète, fiable, sécurisée, évolutive, et offrant une expérience utilisateur optimale grâce à l'intégration d'une intelligence artificielle et d'une interface moderne. Ces résultats constituent une base solide pour d'éventuelles évolutions futures, comme l'ajout de paiement en ligne, de système d'avis, ou d'application mobile.

PARTIE II. MISE EN PLACE DE L'ENVIRONNEMENT DE DÉVELOPPEMENT

2.1 Technologies utilisées

Cette section présente les différents outils, bibliothèques et services utilisés pour le développement de la plateforme. Ils sont classés par catégorie afin de mieux comprendre leur rôle et leur utilité dans le projet.

Technologie / Outil	Rôle / Description
Python 3.13	Langage principal du backend, choisi pour sa simplicité, sa lisibilité et sa performance.
FastAPI	Framework web moderne pour concevoir l'API REST avec validation automatique des données.
Uvicorn	Serveur ASGI léger pour exécuter l'application FastAPI en environnement local.
SQLAlchemy	ORM facilitant la manipulation de la base de données via des classes Python.
Alembic	Outil de gestion des migrations de la base de données.
PostgreSQL	Système de gestion de base de données relationnelle utilisé en production et développement.
Pydantic	Validation stricte du format des requêtes et réponses API.
python-jose	Génération et vérification des jetons d'authentification JWT.
passlib[bcrypt]	Hachage sécurisé des mots de passe en base de données.
python-multipart	Gestion de l'envoi de fichiers (ex: photo de profil via multipart/form-data).
pydantic-settings	Chargement automatique des configurations centralisées depuis un fichier .env.
google-generativeai	SDK officiel pour intégrer l'assistant virtuel basé sur l'IA Gemini.
smtplib / email	Bibliothèques standard Python pour la gestion de l'envoi des e-mails automatiques.
python-dotenv	Chargement des variables d'environnement dans le projet.
psycopg2-binary	Pilote adaptateur pour la connexion de Python à PostgreSQL.

Tableau 1 : Technologies utilisées

2.2 Installation et configuration des outils

Cette section décrit l'installation et la configuration des principaux outils utilisés pour le développement de la plateforme. Nous nous concentrons sur les logiciels nécessaires à la mise en place de l'environnement de travail, sans entrer dans le détail du code applicatif, qui sera présenté plus loin. Pour chaque outil, nous expliquons son rôle, les étapes d'installation et les paramètres de configuration importants.

2.2.1 Installation de Python

Python est le langage de programmation principal du backend. Il doit être installé sur la machine de développement avant toute autre chose. La version 3.13 a été utilisée, mais toute version récente

Étapes d'installation :

1. Télécharger l'installateur depuis le site officiel python.org.
2. Lancer le programme d'installation.
3. **Important :** cocher la case "**Add Python to PATH**" avant de cliquer sur "**Install Now**". Cette option permet d'utiliser Python directement depuis le terminal sans spécifier son chemin complet.

2.2.2 Installation des bibliothèques Python

Les dépendances du backend sont installées avec pip.

Commande unique :

```
pip install fastapi uvicorn sqlalchemy alembic psycopg2-binary python-jose[cryptography] passlib[bcrypt] python-multipart pydantic-settings google-generativeai python-dotenv
```

2.2.3 Installation et configuration de PostgreSQL

PostgreSQL est le système de gestion de base de données relationnelle utilisé pour stocker toutes les données de la plateforme : utilisateurs, disponibilités, réservations, notifications, et les autres. Il a été choisi pour sa robustesse, sa conformité aux normes SQL et sa capacité à gérer des volumes importants de données.

2.3 Structure du projet

Après toutes les installations et configurations, voici la structure complète du projet.

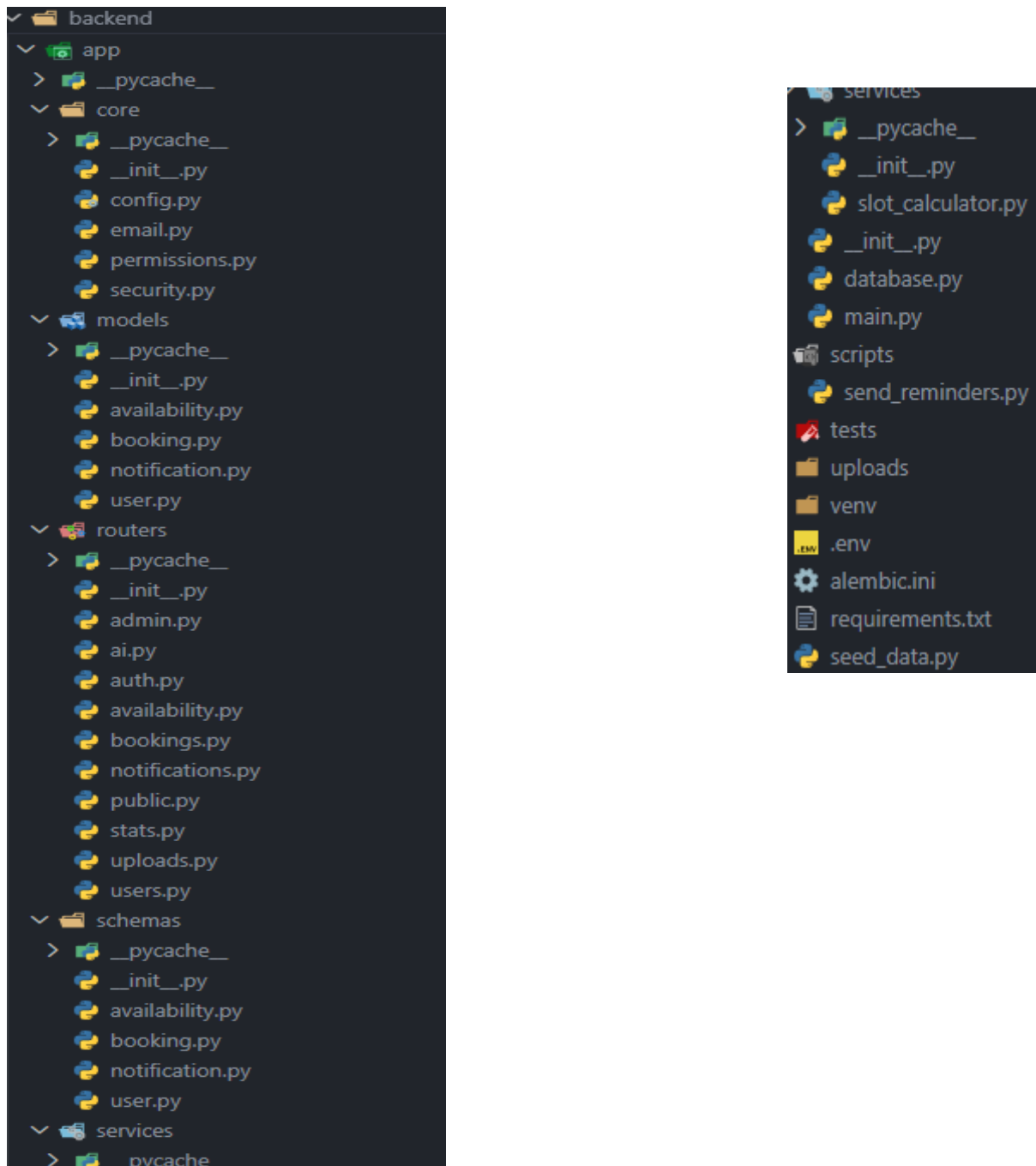


Figure 1 : Structure du projet

2.4 Présentation du code essentiel

Cette section présente les principaux fichiers de code du backend. Pour chaque fichier, nous expliquons son rôle, son fonctionnement et son importance dans l'architecture globale de l'application.

2.4.1 Connexion à la base de données

Le fichier `database.py` est la pierre angulaire de la persistance des données. Il configure la connexion à PostgreSQL via SQLAlchemy et fournit une session de base de données pour chaque requête. Ce module centralise les paramètres de connexion et garantit que toutes les opérations de lecture et d'écriture se font de manière cohérente et sécurisée.

```
ckend / app / database.py
1  from sqlalchemy import create_engine
2  from sqlalchemy.orm import sessionmaker, declarative_base
3  from app.core.config import settings
4
5  engine = create_engine(settings.database_url)
6  SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
7  Base = declarative_base()
8
9  def get_db():
10     db = SessionLocal()
11     try:
12         yield db
13     finally:
14         db.close()
```

Figure 2 : Connexion à la base de données

2.4.2 Configuration

Le fichier `config.py` centralise toutes les variables de configuration sensibles ou modifiables. Il utilise `pydantic-settings` pour charger automatiquement les paramètres depuis le fichier `.env`, ce qui évite de coder en dur des informations comme les identifiants de base de données ou les clés API.

```
backend > app > core > config.py
1  from pydantic_settings import BaseSettings
2
3  class Settings(BaseSettings):
4      database_url: str
5      secret_key: str
6      algorithm: str = "HS256"
7      access_token_expire_minutes: int = 60
8      gemini_api_key: str | None = None
9      smtp_password: str | None = None
10
11     class Config:
12         env_file = ".env"
13         case_sensitive = False
14
15 settings = Settings()
```

Figure 3 : Configuration de l'application

2.4.3 Sécurité

Ce module est responsable de la sécurité de l'application. Il contient les fonctions de hachage des mots de passe avec bcrypt et la gestion des jetons JWT pour l'authentification des utilisateurs.

```
backend > app > core > security.py
1  from datetime import datetime, timedelta, timezone
2  from jose import jwt
3  from passlib.context import CryptContext
4  from app.core.config import settings
5
6  pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
7
8  def hash_password(password: str) -> str:
9      return pwd_context.hash(password)
10
11 def verify_password(plain_password: str, hashed_password: str) -> bool:
12     return pwd_context.verify(plain_password, hashed_password)
13
14 def create_access_token(data: dict) -> str:
15     to_encode = data.copy()
16     expire = datetime.now(timezone.utc) + timedelta(minutes=settings.access_token_expire_minutes)
17     to_encode.update({"exp": expire})
18     return jwt.encode(to_encode, settings.secret_key, algorithm=settings.algorithm)
19
20 def decode_access_token(token: str) -> dict | None:
21     try:
22         return jwt.decode(token, settings.secret_key, algorithms=[settings.algorithm])
23     except jwt.JWTError:
24         return None
25
26
27 from fastapi import Depends, HTTPException, status
28 from fastapi.security import OAuth2PasswordBearer
29 from sqlalchemy.orm import Session
30 from app.database import get_db
31 from app.models.user import User
32
33 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")
34
35 def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)) -> User:
36     credentials_exception = HTTPException(
37         status_code=status.HTTP_401_UNAUTHORIZED,
38         detail="Could not validate credentials",
39         headers={"WWW-Authenticate": "Bearer"},
40     )
41     payload = decode_access_token(token)
42     if payload is None:
43         raise credentials_exception
44     user_id = payload.get("sub")
45     if user_id is None:
46         raise credentials_exception
47     user = db.query(User).filter(User.id == int(user_id)).first()
48     if user is None:
49         raise credentials_exception
50     return user
```

Figure 4 : Sécurité et authentification

2.4.4 Application principale

Le fichier `main.py` est le point d'entrée de l'API FastAPI. Il configure les middlewares (CORS, fichiers statiques), monte le dossier d'upload et inclut tous les routeurs de l'application.

```
ackend / app > main.py
1  from fastapi import FastAPI
2  from fastapi.middleware.cors import CORSMiddleware
3  from fastapi.staticfiles import StaticFiles
4  import os
5  from app.routers import ai
6
7  from app.routers import auth, availability, bookings, public, users, upload
8
9  # Créer le dossier uploads s'il n'existe pas
10 os.makedirs("uploads", exist_ok=True)
11
12 app = FastAPI(title="Booking Platform API")
13
14 app.add_middleware(
15     CORSMiddleware,
16     allow_origins=["http://localhost:5173"],
17     allow_credentials=True,
18     allow_methods=["*"],
19     allow_headers=["*"],
20 )
21
22 app.mount("/uploads", StaticFiles(directory="uploads"), name="uploads")
23
24 app.include_router(auth.router)
25 app.include_router(availability.router)
26 app.include_router(bookings.router)
27 app.include_router(public.router)
28 app.include_router(users.router)
29 app.include_router(uploads.router)
30 app.include_router(notifications.router)
31 app.include_router(admin.router)
32 app.include_router(stats.router) # <-- AJOUT IMPORTANT
33 app.include_router(ai.router)
34 @app.get("/")
35 def root():
36     return {"status": "ok"}
```

Figure 5 : Application principale

2.4.5 Modèle utilisateur (app/models/user.py)

Le modèle User définit la table users dans la base de données. Il contient toutes les colonnes nécessaires pour stocker les informations des utilisateurs, y compris leur rôle, leurs données de profil et leurs paramètres de service.

```
backend > app > models > user.py
1  from sqlalchemy import Column, Integer, String, DateTime
2  from sqlalchemy.sql import func
3  from app.database import Base
4
5  class User(Base):
6      __tablename__ = "users"
7
8      id = Column(Integer, primary_key=True, index=True)
9      email = Column(String, unique=True, nullable=False, index=True)
10     username = Column(String, unique=True, nullable=False, index=True)
11     password_hash = Column(String, nullable=False)
12     timezone = Column(String, nullable=False, default="UTC")
13     created_at = Column(DateTime(timezone=True), server_default=func.now())
14
15     professional_title = Column(String, nullable=True)
16     bio = Column(String, nullable=True)
17     photo_url = Column(String, nullable=True)
18
19     role = Column(String, nullable=False, default="client")
20
21     # Nouveaux champs pour les professionnels
22     service_type = Column(String, nullable=True)
23     location = Column(String, nullable=True)
```

Figure 6 : Modèle utilisateur

PARTIE III. PRÉSENTATION DE L'APPLICATION

Cette section présente l'interface de l'application et ses différentes fonctionnalités à travers des captures d'écran. Chaque espace réservé indique l'emplacement où insérer l'image correspondante.

3.1 Page d'accueil

La page d'accueil est la première page visible par les visiteurs. Elle permet de rechercher directement un professionnel en saisissant son nom d'utilisateur.

Description :

- Un champ de recherche central permet de trouver un professionnel.
- Un bouton "Voir" déclenche la recherche.
- Un lien en bas de page invite les professionnels à se connecter.

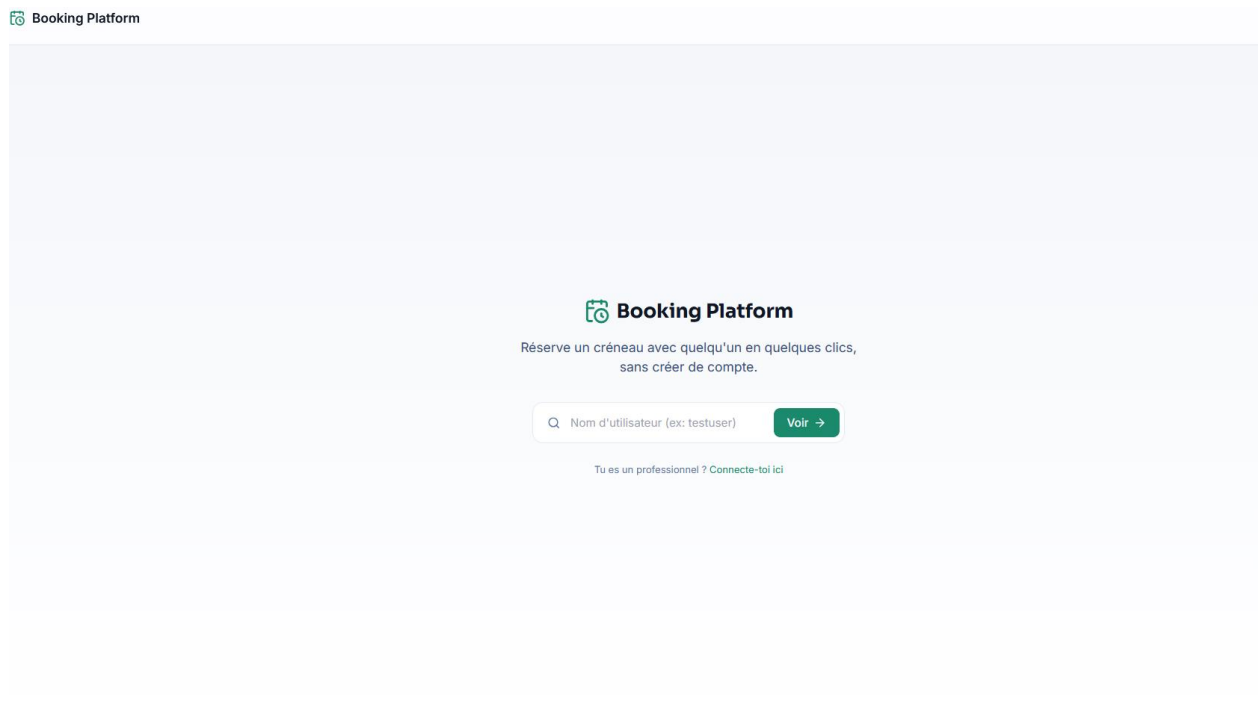


Figure 7 : Page d'accueil

3.2 Page de connexion / inscription

Cette page permet aux utilisateurs de se connecter ou de créer un compte.

Modes :

- Connexion : email et mot de passe.
- Inscription : choix du type de compte (Client ou Professionnel), email, nom d'utilisateur, mot de passe.

Éléments visuels :

- Deux cartes en haut pour choisir le type de compte.
- Formulaire avec icônes dans les champs.
- Bouton de soumission.

-

Booking Platform

Connexion

Heureux de vous revoir !

Adresse email

Mot de passe

Se connecter

Pas de compte ? Créer un compte

-

Figure 8 : Page de connexion

Créer un compte

Rejoignez-nous en quelques secondes

Client

Réserver un créneau

Professionnel

Gérer mes disponibilités

Adresse email

Nom d'utilisateur

Mot de passe

Fuseau horaire détecté : Europe/Bucharest

S'inscrire

Déjà un compte ? Se connecter

Figure 9 : page d inscription

3.3 Espace professionnel (hôte)

3.3.1 Tableau de bord

Le tableau de bord affiche les statistiques de l'hôte.

- Cartes de statistiques : réservations totales, à venir, revenus, taux d'occupation.
- Graphique à barres des réservations par jour de la semaine.
- Bouton d'export CSV.

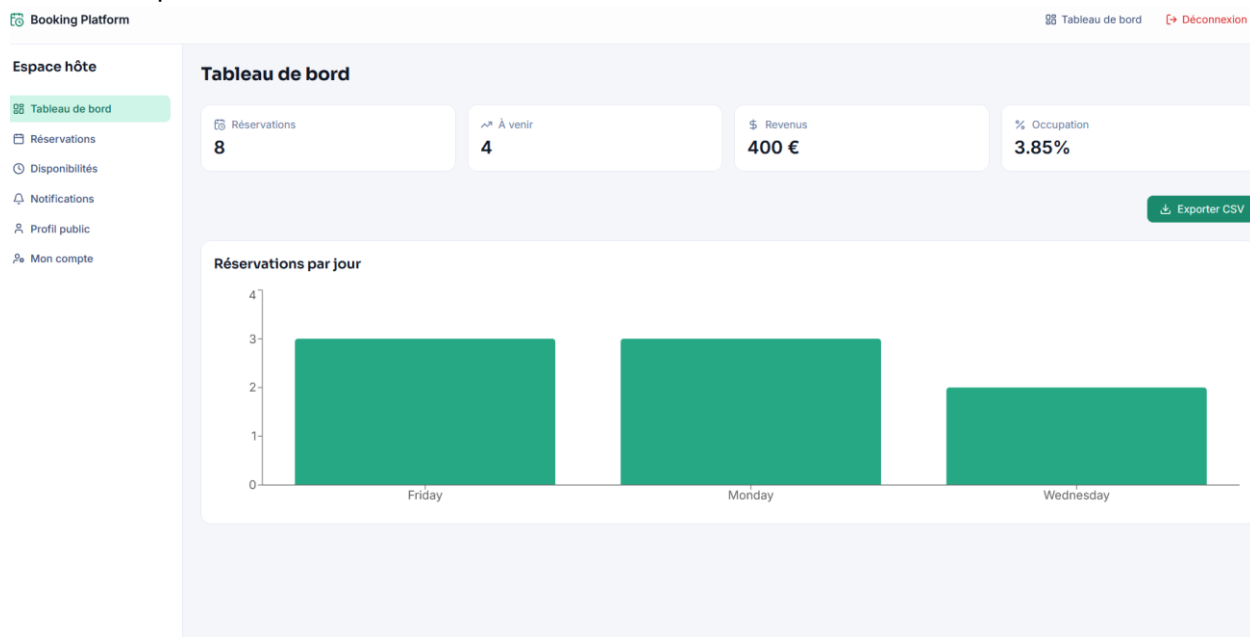


Figure 10 : Tableau de bord professionnel

3.3.2 Gestion des réservations

Liste des réservations avec les informations du client et la possibilité d'annuler.

The screenshot shows the 'Booking Platform' interface. On the left, there's a sidebar titled 'Espace hôte' (Host Space) with navigation links: 'Tableau de bord' (Dashboard), 'Réservations' (Reservations - highlighted), 'Disponibilités' (Availability), 'Notifications', 'Profil public' (Public Profile), and 'Mon compte' (My Account). The main area is titled 'Réservations' and displays a list of reservations. Each reservation entry includes the client's name, email, and a date/time, followed by an 'Annuler' (Cancel) button.

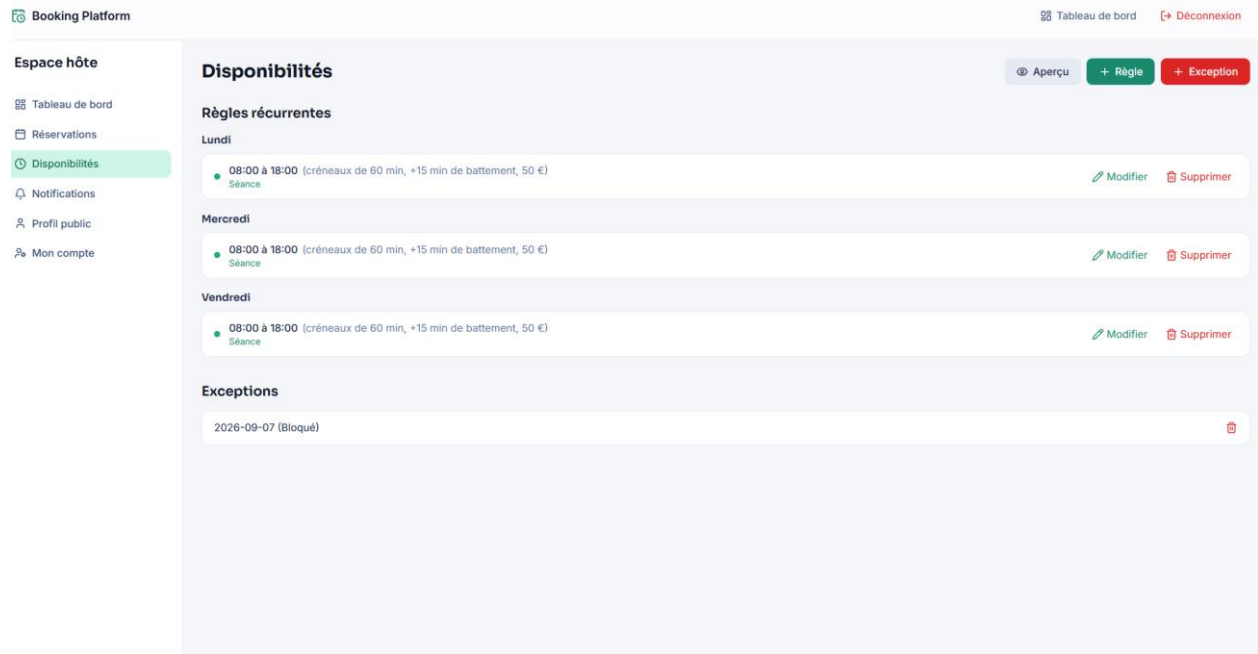
Client	Email	Date/Time	Action
Client 8	client8@example.com	11/09/2026 13:00:00	Annuler
Client 7	client7@example.com	09/09/2026 13:00:00	Annuler
Client 5	client5@example.com	07/09/2026 13:00:00	Annuler
Client 6	client6@example.com	07/09/2026 13:00:00	Annuler
Client 4	client4@example.com	04/09/2026 13:00:00	Annuler
Client 3	client3@example.com	02/09/2026 13:00:00	Annuler
Client 2	client2@example.com	31/08/2026 13:00:00	Annuler
Client 1	client1@example.com	28/08/2026 13:00:00	Annuler

Figure 11 : Gestion des réservations

3.3.3 Gestion des disponibilités

Page permettant de créer, modifier et supprimer des règles de disponibilité.

- Boutons "Règle", "Exception", "Aperçu".
- Liste des règles groupées par jour de la semaine.
- Modale d'ajout/modification avec champs (jour, heure, durée, prix, libellé, battement).
- Aperçu des créneaux disponibles.



Booking Platform

Tableau de bord Déconnexion

Espace hôte

- Tableau de bord
- Réservations
- Disponibilités**
- Notifications
- Profil public
- Mon compte

Disponibilités

Aperçu + Règle + Exception

Règles récurrentes

Lundi

08:00 à 18:00 (créneaux de 60 min, +15 min de battement, 50 €)
Séance

Modifier Supprimer

Mercredi

08:00 à 18:00 (créneaux de 60 min, +15 min de battement, 50 €)
Séance

Modifier Supprimer

Vendredi

08:00 à 18:00 (créneaux de 60 min, +15 min de battement, 50 €)
Séance

Modifier Supprimer

Exceptions

2026-09-07 (Bloqué)

Figure 12 : Gestion des disponibilités

3.3.4 Notifications

Les notifications informent l'hôte des nouvelles réservations ou annulations.

The screenshot shows the 'Booking Platform' interface. At the top, there is a header with 'Booking Platform' on the left and 'Tableau de bord' and 'Déconnexion' on the right. A sidebar on the left, titled 'Espace hôte', contains links to 'Tableau de bord', 'Réservations', 'Disponibilités', 'Notifications' (highlighted in green), 'Profil public', and 'Mon compte'. The main content area is titled 'Notifications' and displays a list of four notifications, each stating 'Nouvelle réservation de Client X le Y/Z/2026 13:00' followed by a '✓ Marquer lue' link.

Notifications	
Nouvelle réservation de Client 5 le 07/09/2026 13:00	✓ Marquer lue
Nouvelle réservation de Client 6 le 07/09/2026 13:00	✓ Marquer lue
Nouvelle réservation de Client 7 le 09/09/2026 13:00	✓ Marquer lue
Nouvelle réservation de Client 8 le 11/09/2026 13:00	✓ Marquer lue

Figure 13 : Notifications du professionnel

3.3.5 Profil public

L'hôte peut modifier son titre professionnel, sa bio, sa photo, son type de service et sa localisation. Un lien public est disponible pour partager sa page.

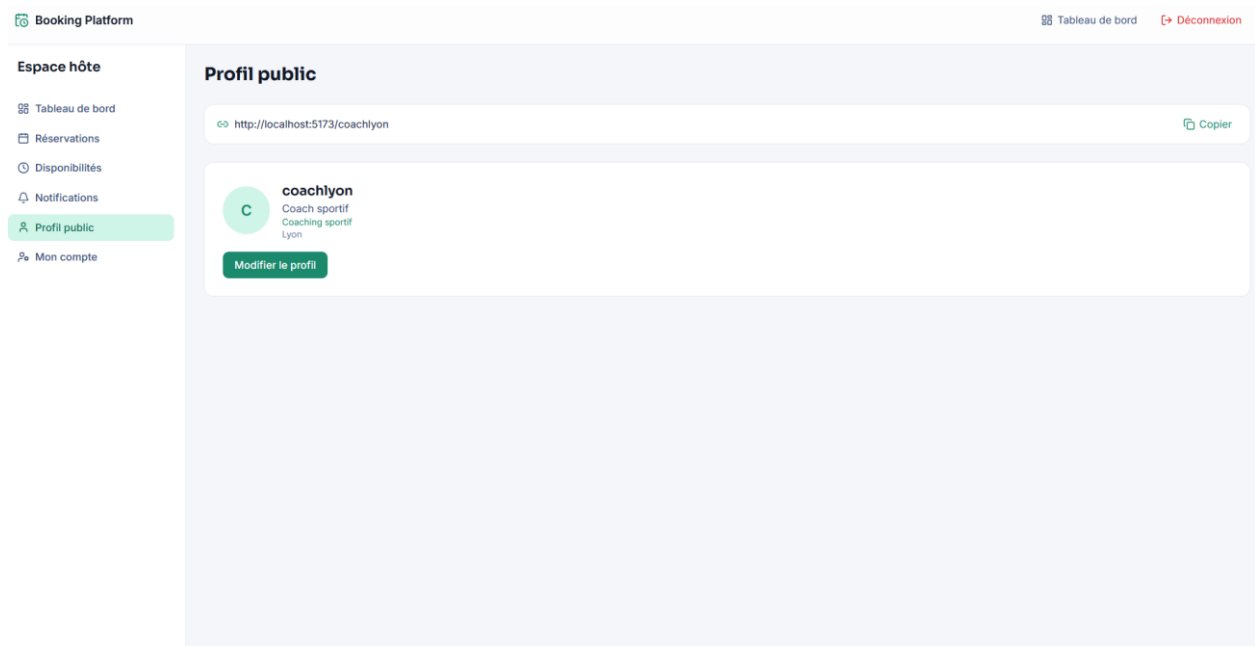


Figure 14 : Profil public du professionnel

3.4 Espace client

3.4.1 Tableau de bord

Le client voit un résumé de son activité : nombre de réservations, rendez-vous à venir, dernières réservations.

Booking Platform Déconnexion

Espace client

- Tableau de bord
- Mes réservations
- Professionnels
- Notifications
- Assistant IA
- Mon compte

Mon espace

Réservations totales: **5** À venir: **2**

Dernières réservations

dentistelyon	09/09/2026 12:00:00	40 € confirmed
pianoparis	08/09/2026 19:00:00	45 € confirmed
coiffeusemarseille	02/09/2026 18:00:00	35 € confirmed
avocatlyon	31/08/2026 18:00:00	150 € confirmed
coachlyon	28/08/2026 13:00:00	50 € confirmed

Trouver un professionnel →

Figure 15 : Tableau de bord client

3.4.2 Mes réservations

Liste des réservations du client avec possibilité d'annulation.

Booking Platform

Deconnexion

Espace client

Tableau de bord

Mes réservations

Professionnels

Notifications

Assistant IA

Mon compte

Mes réservations

Client 12

client1@example.com

09/09/2026 12:00:00

Annuler

Client 14

client1@example.com

08/09/2026 19:00:00

Annuler

Client 11

client1@example.com

02/09/2026 18:00:00

Annuler

Client 15

client1@example.com

31/08/2026 18:00:00

Annuler

Client 1

client1@example.com

28/08/2026 13:00:00

Annuler

Figure 16 : Mes réservations

3.4.3 Recherche de professionnels

Page avec barre de recherche, filtres (type de service, localisation), tri, et cartes des professionnels.

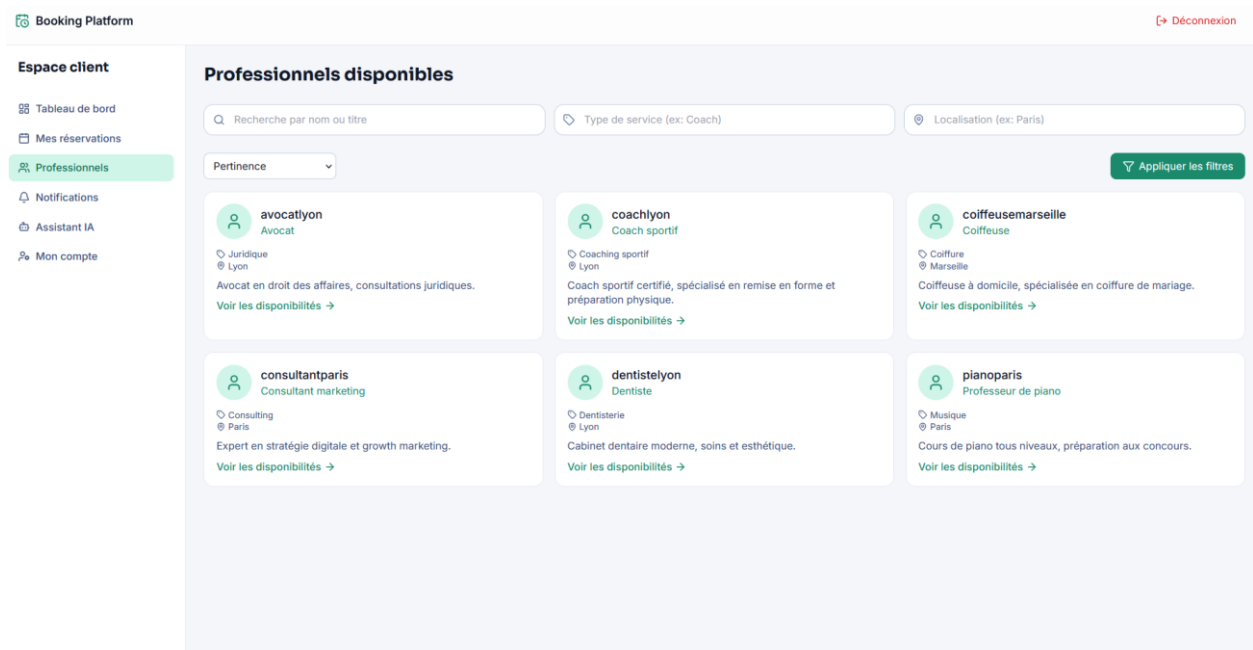


Figure 17 : Recherche de professionnels

3.4.4 Page de réservation

Lorsqu'un client sélectionne un professionnel, il voit :

- Le profil du professionnel (photo, titre, bio, service, localisation).
- Les créneaux disponibles groupés par jour avec onglets.
- Le formulaire de réservation pré-rempli si le client me est connecté.

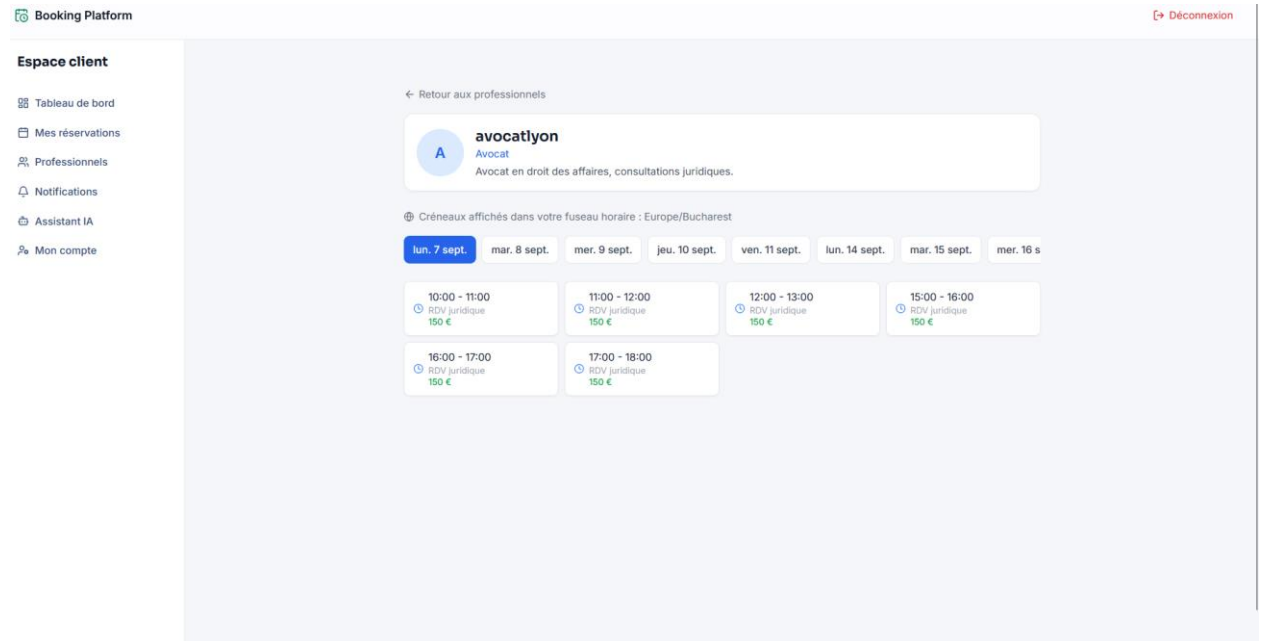


Figure 18 : Page de réservation

3.4.5 Assistant IA

Interface de chat avec l'assistant IA

- Zone de conversation avec bulles.
- Champ de saisie et bouton d'envoi.

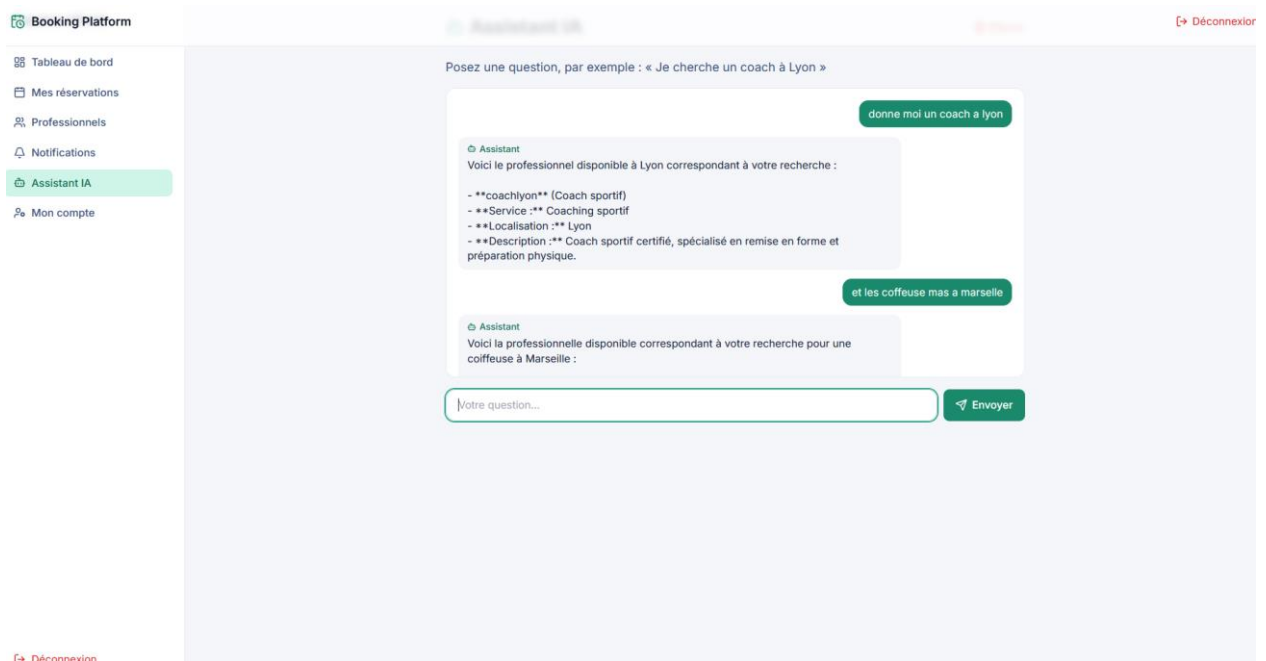


Figure 19 : Assistant IA

3.4.6 Notifications

Le client reçoit des notifications internes (ex : annulation par l'hôte).

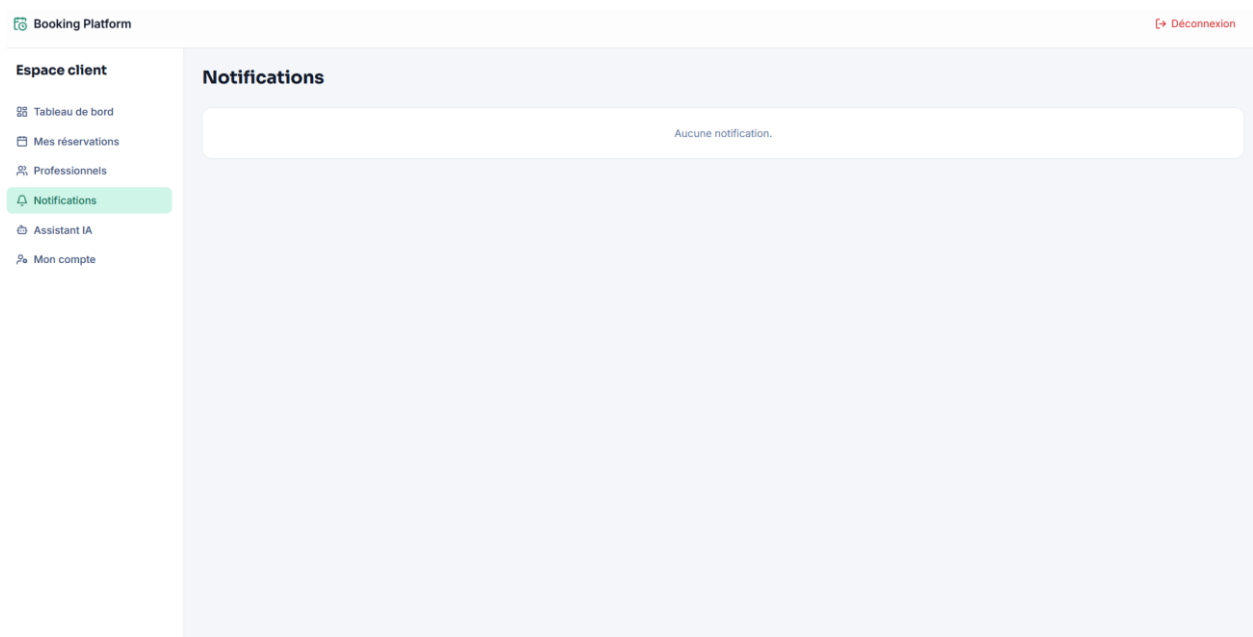


Figure 20 : Notifications du client

3.5 Espace administrateur

3.5.1 Tableau de bord

Statistiques globales : utilisateurs, hôtes, clients, réservations, revenus, graphique.

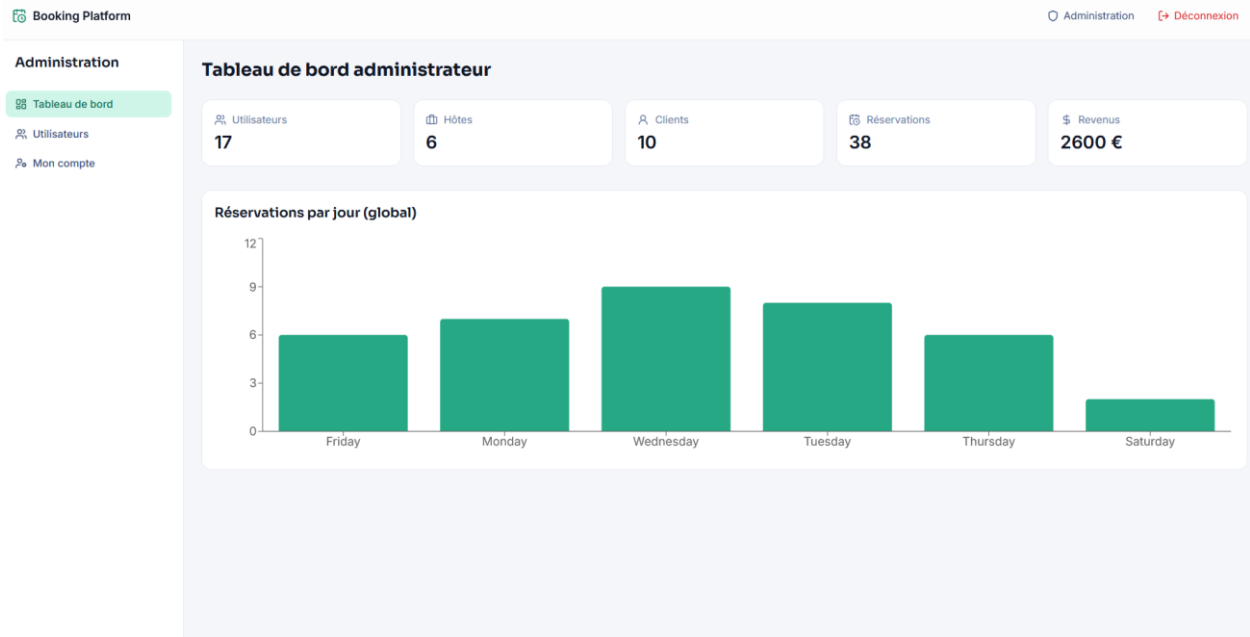


Figure 21 : Tableau de bord administrateur

3.5.2 Gestion des utilisateurs

Tableau listant tous les utilisateurs avec leur rôle. L'admin peut modifier le rôle ou supprimer un compte.

Booking Platform

Administration

Déconnexion

Administration

Tableau de bord

Utilisateurs

Mon compte

Gestion des utilisateurs

Nom	Email	Rôle	Actions
admin	admin@example.com	admin	
coachlyon	coach.lyon@example.com	host	
consultantparis	consultant.paris@example.com	host	
coiffeusemarseille	coiffeuse.marseille@example.com	host	
dentistelyon	dentiste.lyon@example.com	host	
pianoparis	professeur.piano@example.com	host	
avocatlyon	avocat.lyon@example.com	host	
client1	client1@example.com	client	
client2	client2@example.com	client	
client3	client3@example.com	client	
client4	client4@example.com	client	
client5	client5@example.com	client	
client6	client6@example.com	client	
client7	client7@example.com	client	
client8	client8@example.com	client	
client9	client9@example.com	client	

Figure 22 : Gestion des utilisateurs

CONCLUSION

Au terme de ce projet, nous avons conçu et réalisé une **plateforme de réservation intelligente** destinée à simplifier et moderniser la mise en relation entre professionnels et clients. L'objectif principal était de remplacer les méthodes traditionnelles de prise de rendez-vous, souvent manuelles et sources d'erreurs, par une solution centralisée, fiable et accessible en ligne.

La plateforme offre aux professionnels un espace complet pour gérer leurs disponibilités, définir leurs tarifs, leurs types de services et leurs exceptions. Les clients disposent d'un espace intuitif pour rechercher un professionnel, consulter les créneaux disponibles en temps réel et réserver en quelques clics. L'administrateur supervise l'ensemble des utilisateurs et accède à des statistiques globales. Cette séparation en trois rôles garantit une organisation claire et adaptée à chaque profil.

L'intégration d'un **assistant intelligent** basé sur Google Gemini constitue une valeur ajoutée significative. En permettant aux clients de formuler leurs besoins en langage naturel, l'assistant facilite la recherche de professionnels et enrichit l'expérience utilisateur. Cette fonctionnalité illustre la volonté d'exploiter les technologies actuelles pour offrir un service plus pertinent et plus accessible.

Sur le plan technique, l'application repose sur une **architecture modulaire et évolutive**. Le backend, développé avec FastAPI et Python, expose une API REST sécurisée, tandis que le frontend, construit avec React, propose une interface moderne et cohérente. La base de données PostgreSQL assure une persistance fiable, et l'authentification JWT protège les accès. Cette séparation des couches facilite la maintenance et permet d'envisager des extensions futures sans remettre en cause les fondations.

Ce projet démontre qu'il est possible de développer une solution complète et fonctionnelle en utilisant des technologies modernes. Il constitue une base solide pour l'ajout de nouvelles fonctionnalités telles que le paiement en ligne, la gestion des avis, l'envoi de rappels par SMS ou le développement d'une application mobile. La démarche méthodologique adoptée — analyse, conception, réalisation — a garanti la cohérence et la fiabilité de l'ensemble.

En définitive, cette plateforme contribue concrètement à la digitalisation des services de réservation, avec un impact positif sur l'efficacité des professionnels et la satisfaction des clients. Elle confirme le rôle central des technologies de l'information dans la modernisation des processus et ouvre la voie à des améliorations continues pour en faire un outil encore plus performant et complet.

GLOSSAIRE

API (Application Programming Interface) : Interface de programmation permettant à des applications de communiquer entre elles et d'échanger des données de manière structurée.

Backend : Partie serveur d'une application, responsable de la logique métier, de la gestion des données et de la sécurité.

Frontend : Partie client d'une application, correspondant à l'interface utilisateur avec laquelle interagit directement l'utilisateur.

JWT (JSON Web Token) : Standard de jeton d'authentification sécurisé, utilisé pour transmettre des informations entre le client et le serveur de manière fiable et compacte.

ORM (Object-Relational Mapping) : Technique de programmation permettant de manipuler une base de données relationnelle à l'aide d'objets, en évitant l'écriture manuelle de requêtes SQL.

API REST : Style d'architecture basé sur des principes de communication HTTP, où les données sont exposées sous forme de ressources accessibles par des URL.

Base de données PostgreSQL : Système de gestion de base de données relationnelle open-source, reconnu pour sa robustesse et sa conformité aux standards SQL.

IA (Intelligence Artificielle) : Ensemble de techniques visant à simuler des capacités cognitives humaines, comme la compréhension du langage naturel.

Google Gemini : Modèle d'intelligence artificielle générative développé par Google, capable de générer des réponses en langage naturel à partir de requêtes textuelles.

Prompt : Texte d'entrée fourni à un modèle d'IA pour obtenir une réponse ou une action spécifique.

RAG (Retrieval-Augmented Generation) : Approche combinant la recherche d'informations dans une base de connaissances et la génération de texte par un modèle d'IA.

Pydantic : Bibliothèque Python utilisée pour la validation et la gestion des données, intégrée à FastAPI pour définir des schémas stricts.

Tailwind CSS : Framework CSS utilitaire permettant de construire rapidement des interfaces utilisateur cohérentes et responsives.

React : Bibliothèque JavaScript pour la construction d'interfaces utilisateur interactives, basée sur des composants réutilisables.

ANNEXES

Annexe 1 : Extraits de code significatifs

Cette annexe contient les fichiers de code les plus importants du projet :

- Configuration de l'application (main.py)
- Modèle utilisateur (user.py)
- Sécurité et authentification (security.py)
- Calcul des créneaux disponibles (slot_calculator.py)
- Routeur de réservation (bookings.py)
- Routeur de l'assistant IA (ai.py)
- Composant de tableau de bord (DashboardPage.jsx)
- Composant de recherche de professionnels (ProfessionalsPage.jsx)

(Insérer les extraits de code pertinents)

Annexe 2 : Modèle de données

Cette annexe décrit la structure de la base de données :

- Table users : liste des colonnes (id, email, username, role, etc.)
- Table availability_rules : règles de disponibilité
- Table availability_exceptions : exceptions (congrés, fermetures)
- Table bookings : réservations
- Table notifications : notifications internes

(Insérer le schéma ou le dictionnaire de données)

Annexe 3 : Guide d'utilisation rapide

Cette annexe fournit un guide rapide pour les utilisateurs :

- **Professionnel** : comment créer une règle, bloquer une date, voir ses réservations.
- **Client** : comment rechercher un professionnel, réserver un créneau, annuler.
- **Administrateur** : comment changer un rôle, supprimer un utilisateur, consulter les statistiques.

(Insérer le guide d'utilisation)

Annexe 4 : Script de données de démonstration

Cette annexe présente le script seed_data.py qui permet de peupler la base avec des données fictives pour tester la plateforme. Il inclut des exemples d'utilisateurs, de règles, de réservations et de notifications.

